

GitHub Copilot app — the standalone desktop client's own config surface

Written 2026-09-22, split out of `copilot-desktop-apps-coverage-gap.html` 's four-surfaces disambiguation into its own doc, the same way VS Code's built-in Copilot Chat extension and Microsoft 365 Copilot already were. This is the third of those four surfaces: **the GitHub Copilot app** (`github.com/features/ai/github-app`) — a brand-new, separate desktop application, described on GitHub's own page as *"the only desktop experience for agent-driven development built natively on GitHub."* Not the classic GitHub Desktop git client, not the `copilot` CLI, and not VS Code's Copilot Chat extension — a fourth, distinct product that happens to share the "Copilot" name and, confusingly, the same `~/.copilot` home-directory name the CLI adapter already targets.

Where this stands: never targeted by this codebase's existing adapter. Bringing it into scope is possible in principle — a real, per-device configuration surface exists — but every path to using it carries a distinct kind of risk from anything else this investigation has covered: not a network-protocol mismatch, but a live, undocumented, actively-written SQLite database sitting behind an app that isn't open source.

What was actually found (live install, 2026-09-15)

`/Applications/GitHub Copilot.app`, home directory `~/.copilot/` — sharing a name with the CLI's own home directory, which is itself a little misleading, since the two products don't share files.

The files this codebase's existing adapter writes to don't exist on this real, actively-used install at all:

```
~/.copilot/
├─ config.json           # app metadata only – NOT settings.json
├─ data.db (+ -shm, -wal) # SQLite, actively open – real config lives here
├─ session-store.db
├─ repo-metadata-cache.db
├─ servers/              # empty
├─ ide/                  # empty
├─ installed-plugins/    # empty
└─ logs/, run/, session-state/, hooks/, media-cache/
```

No `mcp-config.json`! No `settings.json`. `config.json` only carries UI/app metadata (`appTipShown`, `reasoningSummariesCleanupDone`, ...) — nothing resembling an MCP registry or an LLM redirect.

Where the real config actually lives: `data.db` (SQLite)

Two tables matter for the LLM leg:

```
CREATE TABLE model_providers (  
  id TEXT PRIMARY KEY NOT NULL,  
  name TEXT NOT NULL,  
  type TEXT NOT NULL DEFAULT 'openai',  
  settings_json TEXT NOT NULL DEFAULT '{}',  
  account_id TEXT REFERENCES accounts(id) ON DELETE CASCADE,  
  -- + created_at, updated_at  
);  
  
CREATE TABLE provider_models (  
  id TEXT PRIMARY KEY NOT NULL,  
  provider_id TEXT NOT NULL REFERENCES model_providers(id) ON DELETE CASCADE,  
  model_id TEXT NOT NULL,  
  wire_model TEXT,  
  -- + display_name, max_prompt_tokens, max_output_tokens, wire_api_override, ...  
);
```

The live row, on the install this was checked against:

```
model_providers:  
  id           = github_copilot:eb941f95-68df-4c6a-873d-5a3f4c959166  
  name         = GitHub Copilot  
  type        = github_copilot  
  settings_json = {"authKind":"none","baseUrl":"","  
                  "headersJson":"{}","wireApi":"responses"}  
  account_id   = eb941f95-68df-4c6a-873d-5a3f4c959166
```

`baseUrl` is the field. It's a JSON key, inside a TEXT column, inside a SQLite row — not a file this codebase's `apply_config_delta` /JSON-merge machinery can touch. `provider_models` was empty (0 rows) on the install checked.

No MCP-server-definition table exists anywhere in the schema. The only MCP-adjacent tables are session-scoped event logs (`session_managed_mcp_event_display_names`, `session_mcp_event_sources`) — tracking, not configuration. This lines up with an unverified public search finding that MCP config for GitHub's newer surfaces may have moved to a per-project file (`<project>/copilot/mcp-config.json`) rather than a per-user-home file — not confirmed against this exact app; worktree checkouts inspected during this investigation had no such file, but those may not be representative.

Re-verified directly against the app's compiled binary (`strings` on `/Applications/GitHub Copilot.app/Contents/MacOS/github`, v1.1.21 — not guessed), following up on a GitHub changelog claiming the app supports BYOK ("Settings → Model Providers", 2026-06-23).

⚠ **Evidence class, stated plainly:** everything below came from `strings` on a compiled binary, not from a formally issued GitHub document or public source code — this app isn't open source. The embedded SQL text is real (it's the literal shipped code, not a paraphrase), so the *schema itself* can be trusted as accurate. But nothing here is GitHub-published or GitHub-supported: no commitment it stays stable across an update.

- **The binary's embedded SQL migration history is readable, and it names the design.** A code comment reads: *"BYOK (Bring Your Own Key) tables. Stores user-configured model providers and the models they expose to the app... See plan in docs/byok.md... for the full design."* This is a real, first-party, named feature — not a coincidental field.
- **`model_providers` is literally the same table as `byok_providers`.** A later migration in the same embedded history runs `ALTER TABLE byok_providers RENAME TO model_providers`. The `baseUrl` field found above is the BYOK storage, post-rename. **The full pre-rename schema is recoverable from the binary:**

```
CREATE TABLE byok_providers (  
    id TEXT PRIMARY KEY NOT NULL,  
    name TEXT NOT NULL,  
    type TEXT NOT NULL CHECK (type IN  
        ('openai','azure','anthropic','ollama','foundry_local','custom')),  
    base_url TEXT NOT NULL,  
    wire_api TEXT NOT NULL DEFAULT 'completions'  
        CHECK (wire_api IN ('completions','responses')),  
    auth_kind TEXT NOT NULL DEFAULT 'api_key'  
        CHECK (auth_kind IN ('api_key','bearer_token','none')),  
    headers_json TEXT NOT NULL DEFAULT '{}',  
    -- + created_at, updated_at  
);  
CREATE TABLE byok_models (  
    id TEXT PRIMARY KEY NOT NULL,  
    provider_id TEXT NOT NULL REFERENCES byok_providers(id) ON DELETE CASCADE,  
    model_id TEXT NOT NULL,  
    -- + wire_model, display_name, max_prompt_tokens, max_output_tokens, ...  
);
```

`type='custom'` with `auth_kind='none'` is exactly the shape a local, no-auth endpoint needs. A session then selects it via a `sessions.model = 'byok:<provider_id>:<model_id>'` string, confirmed from the same embedded SQL. Secrets are kept out of the DB entirely, in the OS keychain under `byok:<provider_id>:apiKey` / `...:bearerToken`.

A real gap in the evidence, worth stating plainly rather than smoothing over

An earlier pass of this investigation treated the URL-override behavior as "*confirmed at the runtime code level, not just the schema*" — but the source actually read for that confirmation was `openAIProvider.ts` / `anthropicProvider.ts` inside `microsoft/vscode`'s bundled Copilot extension: **a different product**, VS Code's own separate BYOK feature, not this standalone app's code. That source does show a real, non-hardcoded `model.url` driving the actual completion request rather than a fixed default — a genuine, useful finding — but it's evidence about VS Code's BYOK implementation, not this app's. The two products share a name for the feature and a broadly similar schema shape, which is almost certainly why they got conflated; it doesn't mean they share code. **For the standalone GitHub Copilot app specifically, only the schema is confirmed** (the binary's embedded SQL, read directly) — whether a `type='custom'` provider's real inference call actually uses the stored `base_url` rather than some hardcoded default remains schema-plausible, not runtime-confirmed, for this app. Closing that gap would need reading this app's own decompiled request-building logic, or a live test with a real `custom` provider row and a listener to observe whether it's actually hit.

Update, 2026-09-24: a second static pass strengthens this, but doesn't close it

Went back to `strings` on the same binary (v1.1.21) looking specifically for this app's *own* request-building code, not VS Code's. Found substantially more than the first pass, without fully closing the gap.

New architectural fact: the app is built with **Tauri** — a Rust backend paired with `wry` (which renders through WKWebView on macOS, consistent with the WKWebView pattern found everywhere else in this investigation's Mac-side findings, not a coincidence).

The real internal module structure is recoverable from embedded Rust panic-location strings — genuine source file paths compiled into the binary, not guessed:

```
model_providers/mod.rs           - generic provider logic (675+ lines)
model_providers/kinds/github_copilot.rs - GitHub's own first-party path
model_providers/azure_cli.rs      - Azure-specific handling
model_providers/local/process_registry.rs - local model processes (Ollama)
```

The strongest new evidence: the binary contains a set of literal endpoint-path fragments consistent with an `AssistantUsageApiEndpoint`-shaped enum — `/chat/completions`, `/v1/messages`, `/responses`, and even a `ws:/responses` WebSocket variant. That's a real, selectable set of endpoint *suffixes*, correlating directly with the already-confirmed `wireApi` schema field (`'completions'|'responses'`). The only reason an app needs multiple selectable suffix variants at all is to append one of them to *something* at request time — structurally consistent with base-plus-suffix URL construction, though not a literal proof of it.

What this still doesn't show: the actual concatenation happening in code. Rust's `format!` macro splits literal fragments at compile time in a way `strings` can't reassemble into "which variable gets joined with which literal" — that needs real disassembly, not available in this pass.

Net: meaningfully stronger than the first pass — a real, app-specific structural signal (the endpoint-suffix enum, the confirmed module split between generic/Azure/local/first-party provider handling) rather than an inference borrowed from a different product. Still short of proof. The schema-vs-runtime distinction in the paragraph above stands; this update narrows it without closing it.

Update, 2026-09-24 (live test): the gap is closed — confirmed directly, not inferred

Ran the live test the previous update said was the only path left. On a real, currently-used installation of the app (v1.1.22):

1. Quit the app; took a full, verified backup of `~/.copilot/` (`diff -rq` confirmed byte-identical).
2. Re-checked the live schema directly against the running install's `data.db` rather than trusting the earlier snapshot — a real, deliberate catch: the app auto-updated itself (1.1.21 → 1.1.22, via its own Tauri updater) partway through this investigation, and the CHECK constraint on `model_providers.type` that an earlier pass found is **gone** in the current schema — `type` is now unconstrained. Exactly the "accept re-validation on every app update" risk this doc names elsewhere, caught in the act rather than assumed away.
3. Inserted one `model_providers` row (`type: 'custom'`, `authKind: 'none'`, `baseUrl` pointed at a local listener) and one matching `provider_models` row, with the app fully closed — no concurrent writer.
4. Relaunched, confirmed the row surfaced correctly in Settings → Model providers (matching what was written, byte for byte), selected it as the active model in a plain chat (not a Project/agent session — those lock to GitHub's own first-party models, e.g. `mai-code-1.1-flash`, and never reach a custom provider at all), and sent a real prompt.

Result: the real inference call hit the listener. The chat displayed this test's own fixed placeholder text verbatim, and the listener logged the actual `POST` that produced it. **The runtime-confirmation gap is closed — a `type='custom'` provider's real inference call does read the stored `base_url`, for this app specifically, not just VS Code's separate implementation.**

Three further findings from the real request, beyond the core question:

- **The app calls out to the official OpenAI Node.js SDK for BYOK completions**, not a direct Rust HTTP call. The request carried `x-stainless-lang: js`, `x-stainless-runtime: node`, and `user-agent: OpenAI/JS 5.20.1` — the fingerprint of OpenAI's own official JS client library. This Tauri/Rust app shells out to (or embeds) a Node runtime specifically for this path.
- **`authKind: 'none'` behaves exactly as the schema implied:** the request carried `authorization: Bearer` — present, empty — confirmed live, not assumed from the field's

name.

- **The full first-party agent system prompt is sent to whatever `base_url` names, unmodified.** The real request body (126 KB) carried GitHub's own internal coding-agent system prompt — real tool-use and sub-agent rules, not sanitized or stripped before being sent to a user-configured third-party endpoint. Not quoted at length here since it's GitHub's own proprietary prompt content, but worth recording as a real behavior: pointing a custom provider at an observing proxy would see that prompt in full, every time.

Cleanup: quit the app again, deleted the two inserted rows (not a full backup restore, since real chat history had legitimately accumulated during the test and there was no reason to discard it), relaunched, and confirmed `model_providers` / `provider_models` were back to their exact pre-test state (one row, zero rows respectively). The verified backup (`~/.copilot.backup-pretest-2026-09-24`) was never needed but stayed available throughout.

Two distinct BYOK mechanisms exist — only one is reachable from a local proxy

GitHub's enterprise admin console has its own, separate "Configure custom models" flow (AI controls → Copilot), confirmed **entirely server-side**: the admin's key and endpoint are held by GitHub's platform and pushed down to clients through it. That one is useless for a local interception approach — GitHub's servers can't reach a `127.0.0.1` loopback listener. The relevant one is the **per-device** `byok_providers` table above, confirmed to be "global per app install" — evaluated locally, by the app itself.

Correction, 2026-09-24, against GitHub's own docs source (`github/docs` , `content/copilot/how-tos/github-copilot-app/use-byok-models.md` , read verbatim via `gh api` , not a rendered/summarized page): the earlier "works without a signed-in GitHub account" claim is **wrong**. The doc states plainly: "*You must sign in with a GitHub account to use the app*" — the part that's actually optional is the **Copilot plan**, not the account: "*you do not need a Copilot plan if you use your own model provider.*"

That same source also confirms two things worth having in writing rather than inferred from the schema: the per-device provider list is **documented**, not just schema-recovered — OpenAI, Azure OpenAI, Microsoft Foundry, Anthropic, Ollama, Foundry Local, LM Studio, and **"Any OpenAI-compatible HTTP endpoint"** — and the add-provider form fields are described as "*the display name, base URL, and API key*" (varies by provider). So the `type='custom'` + `base_url` row this doc's live test used is the **documented, supported shape** for the OpenAI-compatible case, not a reverse-engineered one — still explicitly **public preview**, **"subject to change,"** per the same page.

No file, CLI, or programmatic path is documented for the per-device version — GUI-only (`Settings` → `Model providers` → `Add provider`). Automating it without user interaction means either a direct DB write or driving the actual GUI once per device.

The enterprise flow's actual parameters, and where inference calls really go

Enumerated from GitHub's admin how-to doc (`enable-custom-models`), every field the flow exposes:

Provider	Supported	URL configurable?
OpenAI, Anthropic, xAI	Yes — key entered, then a "fetch models" button queries the provider server-side	No URL field described — implies a fixed, known endpoint per provider
AWS Bedrock, Google AI Studio	Listed as supported	Not detailed in what this doc surfaced
Microsoft Foundry	Yes	Yes — explicit "Deployment URL" field, manually entered, plus manual per-model "Model ID" entries (no auto-fetch)
OpenAI-compatible providers	Listed as supported	Confirmed documentation gap, now verified against raw doc source — this category needs <i>some</i> base-URL field by definition, but <code>data/reusables/copilot/byok-add.md</code> in <code>github/docs</code> (read verbatim, not summarized) walks through exactly two branches — "fetch models" for OpenAI/Anthropic/xAI, and a "Deployment URL" field for Microsoft Foundry — and gives OpenAI-compatible no field of its own in either branch , despite listing it as a supported provider one section earlier. Real gap in GitHub's own docs, not a fetch-tooling artifact.

Other fields: **Name** (shown in the model picker), **API Key** (with an explicit least-privilege-scoping recommendation), and **Access** scoping ("Allow for all organizations" or per-organization).

Whether actual inference calls get relayed through GitHub's own backend is architecture-inferred, not directly confirmed by any doc — held to a lower confidence bar than the schema findings above:

- **Enterprise custom models:** very likely relayed through GitHub's CAPI backend for every real inference call, not just the one-time model-list fetch. Basis: the provider API key is entered once, centrally, and never described as being handed to client machines — a client that never holds the credential can't call the provider directly, so GitHub's own backend is almost certainly the one making that outbound call and relaying the response. Reasoned from the security model, not a quoted statement that calls are proxied.

- **Personal/per-device BYOK** (the `byok_providers` table): the opposite architecture — the key lives in the local OS keychain (a GitHub sign-in is still required to use the app itself, corrected above, but no Copilot *plan* is needed for this path). No reason for GitHub's servers to be in the loop; the client almost certainly calls the configured provider (or a local `type='custom'` endpoint) **directly** — and this is exactly what the live test above confirmed.
- Either way, this doesn't change the operative conclusion: the enterprise flow is unreachable from a local proxy because the calling machine is GitHub's servers, not the endpoint; the per-device flow is reachable in principle precisely because the call originates locally — modulo the runtime-confirmation gap noted above.
- **A real dependency for the per-device path:** the two BYOK tiers are additive by default (enterprise-pushed models and personal BYOK entries coexist in the same model picker), but there's a one-directional admin kill-switch specifically over the personal side. If an org has disabled it, personal BYOK is unavailable **org-wide**, before any DB-write question even applies — worth checking that policy's state as an early step in any rollout.

Update, 2026-09-24: a genuinely different strategy — become the enterprise's only configured provider, instead of intercepting per device

Everything above targets the **per-device** BYOK path because that's the one a local proxy can reach. But the enterprise flow's reachability problem (GitHub's servers, not the endpoint, make the call) only rules it out for *local* interception — it doesn't rule it out if the "listener" is a real, internet-reachable relay this org already controls (Optimus / the ZAX gateway). Read that way, the enterprise flow isn't a dead end, it's a **fleet-wide alternative to per-device DB writes entirely**.

Confirmed directly from GitHub's docs source (`github/docs` repo, read via `gh api` — raw markdown, not a rendered/summarized page), not inferred:

Both policies needed for this reach the GitHub Copilot app. GitHub's own [supported-surfaces-for-policies](#) reference table (`content/copilot/reference/supported-surfaces-for-policies.md`) marks both "**Configure custom models**" and "**Configure models**" as supported for the Copilot app, alongside IDEs, the CLI, and copilot.com — this isn't a VS Code-only or CLI-only control.

The recipe, per GitHub's own how-tos (`content/copilot/how-tos/administer-copilot/manage-for-enterprise/enable-custom-models.md` , `.../manage-for-organization/manage-default-models.md`):

1. Enterprise owner enables the "**Enable custom models**" policy, then AI controls → Copilot → **Configure custom models** → **Add API key** → provider, name, key, models. The custom model then "appear[s] at the bottom of the model picker, under the enterprise name" for every member of every org in the enterprise (or scoped to specific orgs via the **Access** tab).

2. Enterprise owner disables the GitHub-hosted models one by one under **Models**, and sets the **"Default availability for released models"** policy so unconfigured/new GA models don't auto-enable. Enterprise-level choices are *enforced*, not advisory: an org sees a  shield icon next to a model the enterprise owner has locked, and "cannot change the availability of this model."

Three honest caveats, not smoothed over:

- **The "zero GitHub-hosted models enabled" end state is never explicitly described.** The per-model toggle plus the default-availability policy make it look reachable, but no doc states what a user sees if every built-in model is disabled and only a custom one remains — and the Project/agentic-session model observed during the live test above (`mai-code-1.1-flash`) may not even be a policy-governed catalog entry, since that surface never showed a picker at all.
- **Only Microsoft Foundry has a documented custom-URL field at the enterprise tier too** — same gap as the per-device table above, verified against the same `byok-add.md` source. OpenAI-compatible is a listed supported provider type with no described URL field anywhere in the add-key flow.
- **This is still the server-side flow.** Whatever endpoint gets registered has to be reachable *from GitHub's own infrastructure*, not from a single device — an internet-facing relay, not a loopback listener. It's a materially different architecture from the per-device DB write proven above: one fleet-wide admin-console configuration instead of a per-device write that has to survive every app auto-update.

Why a direct database write still isn't a "wrong path" fix

- **The DB is open and actively written to right now** — the app was running with the WAL file growing during this investigation. Writing into it from outside the app risks corruption, or being silently overwritten by the app's own next commit, unless done with real SQLite transaction discipline (and ideally the app's own cooperation — e.g. writing only while confirmed not mid-write, which nothing in this codebase's current tooling attempts).
- **No existing code path fits.** Every adapter in this repo assumes a mergeable JSON/TOML *file* (`ConfigDelta` → `config_render` → `apply_config_delta` 's file merge). None of that machinery knows how to address a database row.
- **The schema is confirmed from the shipped binary itself, not guessed — but that isn't the same as GitHub-published or GitHub-supported.** This came from `strings` on a private binary, not a formal doc or open-source repo, so nothing commits GitHub to this shape staying the same release to release, and writing outside the app's own code still bypasses whatever validation/side effects the real "Add provider" flow performs.

Recommended paths forward

1. **Don't attempt a live-DB write without explicit product sign-off**, admin privilege notwithstanding. Root/LocalSystem removes the permissions barrier, not the concurrency hazard — the write still races the app's own live process, can be silently overwritten by its next flush, and skips whatever validation/keychain/cache-invalidation the real "Add provider" flow performs. The blast radius is the app's entire local state (sessions, workspaces, worktrees), not a small, single-purpose config file.
2. **The schema risk is real and confirmed, which changes the shape of "new product design work," not its necessity.** Two realistic directions:
 - Build careful, transaction-safe SQLite write tooling against this now-confirmed schema, with the same "narrow trust, confirm against live install, never guess a schema" discipline established elsewhere in this codebase — and still accept re-validation on every app update, since GitHub doesn't publish this schema.
 - Push for a supported, non-GUI way to provision the per-device BYOK provider — GitHub's own docs confirm none exists today.
3. **The runtime-confirmation gap is closed.** A second static pass (2026-09-24) found real, app-specific structural evidence (the endpoint-suffix enum correlating with `wireApi`), and a live test the same day (see the update above) confirmed it directly: a real `type='custom'` row's `base_url` was hit by the app's actual inference call, observed on a listener. Everything above this is now confirmed true for this app specifically, not inferred.
4. **Evaluate the enterprise "custom-only" posture as an alternative to per-device DB writes entirely**, before investing further in #2. It trades a per-device, update-fragile write for one fleet-wide admin console configuration, confirmed by GitHub's own docs to reach the Copilot app — at the cost of needing a real internet-reachable relay (not a loopback listener) and leaving the "disable every GitHub-hosted model" end state unconfirmed. See the update above for the full recipe and caveats.

Open questions

- Is the GitHub Copilot app actually a stated priority for LLM-traffic brokering, given the live-DB risk profile above? If not, this doc is the record of why it's out of scope, not a to-do.
- Does GitHub expose (or plan to expose) any supported external configuration surface for this app? Worth checking their own repo/issue tracker for the app (separate from `github/copilot-cli`) if one exists publicly.
- Does a `type='custom'` provider's inference call actually use the stored `base_url` for *this* app, not just VS Code's separate implementation of the same idea? **Resolved, 2026-09-24 — confirmed by live test.** A real `type='custom'` provider was inserted directly into a live install's `data.db`, selected as the active model in a plain Chat, and a real inference call hit the configured `base_url`. Test rows were removed and the install verified back to its original state afterward. Two verified backups remain on disk (`~/copilot.backup-2026-09-24`),

~/ .copilot.backup-pretest-2026-09-24) as an unused safety net, not as an open restore point.

- New from the live test: is `ModelPolicy` (`allowedModels` , `disableModelInvocation`) a locally-settable config or an enterprise-server-pushed policy, and could it offer a supported way to pin an interception-friendly provider without a direct DB write? **Resolved, 2026-09-24 — enterprise/server-pushed, not local config, not a usable lever.** See the subsection below.
- **X2:** does the enterprise "custom-only" posture (disable every GitHub-hosted model, enable only an enterprise-registered custom provider) actually work end to end for the GitHub Copilot app specifically — reaching Project/agentive sessions and not just plain Chats — and does GitHub's own infra actually relay to an OpenAI-compatible custom endpoint despite the documented-gap URL field? Confirmed from GitHub's own docs source that the two policies involved (**Configure custom models**, **Configure models**) both reach the Copilot app, and the admin-console recipe is real — see the update below — but the "zero built-in models enabled" end state and the OpenAI-compatible URL-field gap are both unconfirmed. Would need either a real enterprise-owner console (not available in this investigation) or a written response from GitHub to close.

Update, 2026-09-24 (X1): `ModelPolicy` is enterprise/server-pushed, not local config

Three pieces of live evidence, not just `strings` :

No local storage exists for it anywhere. Checked every table in `data.db` (79 tables, including the `settings` singleton row), plus `session-store.db` and `repo-metadata-cache.db` — nothing resembling a model-policy/allowlist table. Unlike `model_providers` / `provider_models` (real, locally-writable, proven above), `ModelPolicy` has no on-disk counterpart to write to.

The actual mechanism was caught live, in this install's own running logs, unprompted, under `copilot_runtime::storage::managed_settings` / `managed_settings::api_session` :

```
[managedSettings] device MDM: no policy present on this device
[managedSettings] self-fetch starting for account https://github.com/ukanitkar
[managedSettings] self-fetch complete ...: serverResolution=Live
[managedSettings] server policy: none for this account (404/empty) from https://github.com
[managedSettings] confirmed no policy served from fresh cache (age 2629474ms) from https://github.com
[managedSettings] effective policy resolved: source=none, bypassDisabled=false, serverFetch=...
```

A real two-source resolver: OS-level device MDM policy, and an authenticated per-account fetch to `https://github.com` , cached locally with a TTL (the "age Xms" logging). On this personal, unmanaged account both come back empty, hence `source=none` — the expected shape for an enterprise-admin-pushed policy that simply isn't configured here, not evidence the mechanism is inert.

It already has a real enforcement consequence, seen in the same logs, proving it isn't dead code:

```
[managedSettings] applied: bypass-permissions mode DISABLED by enterprise policy (fail-closed)
[managedSettings] applied: no bypass restriction in force (managed policy absent)
```

Fail-**closed** when the policy can't be determined — restrictive by default, not permissive.

Correction to the original `strings` -only finding: `disableModelInvocation` is not part of `ModelPolicy` at all — it's a **tool**-invocation permission field, paired with `disableUserInvocation` under `AgentCustomization` /rule customization. That was a false adjacency in the raw string dump, the same "`strings`" can't reassemble which variable joins which literal" limitation already named elsewhere in this doc. `allowedModels` appears once, in a session-creation wire message beside `sessionId / clientName / systemMessage` — consistent with being populated *into* a session from this same MDM/server-resolved policy, not read from a user-editable file.

Net for this project: `ModelPolicy` is the same category as the already-documented "Configure custom models" enterprise flow — GitHub-server/MDM-pushed, not a local lever. It offers no supported path to pin an interception-friendly BYOK provider; if anything it's a *restriction* mechanism (narrowing model choice), the opposite direction from what this project wants. It is also fail-closed, so a genuinely enterprise-managed device could plausibly have it interfere with BYOK provider selection too — unconfirmed, since this account has no managed policy to observe that against.